GenAI-Assisted Career and Operating Systems Exploration

Site Reliability Engineer at Google

Group members

No.	Student ID	Full name	Mobile phone	Section
1				
2				
3				
4				
5				

204341 Operating Systems
Final Assignment

05 - 18 October 2026

1 Introduction and Career Interest

This report studies the Site Reliability Engineer (SRE) position at Google as a target for cooperative education (COOP) or job training. It answers the assignment questions in order: what the job involves, which skills and tools it needs, which operating systems, hardware and infrastructure it works with, and how this connects to the Operating Systems course.

Why SRE. Our reason is our own judgment about where computing work is going. It is an opinion, not a verified fact. Today's Generative AI tools help most with writing software. Keeping that software running in production, on real machines and networks, still needs engineers who can tell what is wrong with a live system. We expect this kind of work to stay valuable as Generative AI spreads. We have not tested this expectation against labor-market data, so we treat it as an assumption. The role also fits the course. Google's SRE book says that SRE is what happens when a software engineer is asked to design an operations team [1]. The job therefore joins programming with operating systems, networking and hardware.

Why Google. We chose Google for two reasons. First, it is a large company with a long SRE history. The SRE book's introduction says its author joined Google in 2003 and was tasked with running a "Production Team" of seven engineers [1]. Second, Google publishes its SRE practice openly at `sre.google`: the SRE book, the SRE Workbook, *Building Secure and Reliable Systems* and further articles. This let us check our claims against Google's own words and not only against second-hand summaries.

Limits of this choice. The same sources also limit what we can say. The books are Google's own publications, dated 2017 to 2020, so most of what we report about Google comes from one company's account. They describe Google's practice when they were written, and they say little about the tools Google uses today. We therefore state a claim about Google only where a published source supports it. Where no published source describes Google's internal environment, we describe a typical environment for the position and label it as an assumption or recommendation. The one Google SRE internship that we found is for the EMEA region only (Section 4), so we treat Google as a study target.

2 Selected Job Position and Job Description

The role. Google's SRE book says that SRE teams hire software engineers to run products and to build systems that do work sysadmins would otherwise do, often manually [1]. The SRE Workbook says the basic tenet of SRE is that doing operations well is a software problem, and describes SRE as both a job role and a set of practices [2]. The SRE book also says SRE can be viewed as a specific implementation of DevOps with some idiosyncratic extensions [1]. A Google job advertisement from one team, Software Engineer, Android SRE, says that SRE combines software and systems engineering to build and run large-scale, massively distributed, fault-tolerant systems [3].

Duties. An SRE team is generally responsible for the availability, latency, performance, efficiency, change management, monitoring, emergency response and capacity planning of its services [1, 2]. Google caps operational work (toil) at 50% of an SRE's time, and the rest should go to project work that uses coding skills [1]. The job advertisement lists writing product or system development code, triaging and debugging issues by analyzing their impact on hardware, network or service operations, and taking part in design reviews [3].

Daily and weekly tasks. Operational load falls into three categories: pages, tickets and ongoing operational activities [1]. Being on call is one of the defining characteristics of SRE at Google, and during a shift SREs diagnose, mitigate, fix or escalate incidents and handle nonurgent production duties [2]. After a page is received and acknowledged, the on-call engineer is expected to triage the problem and work toward its resolution, escalating as needed [1]. Typical paging response times are 5 minutes for user-facing or time-critical services and 30 minutes for less time-sensitive systems [1]. SRE teams must write postmortems with a full timeline after significant incidents [1]. The books

describe SRE teams in general. They give no day-by-day schedule for one SRE, and we do not infer one.

Projects. The books describe project work in four groups. The first is internal tools and automation, such as binary rollout mechanisms and monitoring [1]. The second is capacity planning, where Auxon automates the work for services in Google production [1]. The third is onboarding services: the Production Readiness Review (PRR) is a prerequisite for an SRE team to accept responsibility for managing a service in production [1]. The fourth is launch support, where in one example a special team of Launch Coordination Engineers coordinated the groups involved [1]. A Workbook case study also describes automating the repair of failed datacenter line cards [2]. We found no source on projects given to student SREs beyond the general description in the internship posting (Section 4).

Education and qualifications. The SRE book says 50–60% of SREs are hired through the standard procedure for Google Software Engineers. The other 40–50% are very close to those qualifications and also have technical skills that are useful to SRE but rare among software engineers, most commonly UNIX system internals and networking (Layer 1 to Layer 3) [1]. The job advertisement, which is not for an internship, lists as minimum qualifications a Bachelor's degree in Computer Science, a related field or equivalent practical experience, and 2 years of software development experience in one or more programming languages. It prefers a Master's degree and 2 years of experience troubleshooting large-scale distributed systems [3]. We found no source on certificates, a portfolio or open-source experience that would help an applicant.

3 Hard Skills and Soft Skills

Each claim in this section rests on one source, and we did not compare a second source. Almost all sources are Google's own publications, so they show what Google values, not what every SRE employer values.

3.1 Hard skills

- **Operating systems and networking.** UNIX system internals and networking (Layer 1 to Layer 3) are the two most common specialist skills Google seeks in SREs [1]. Google announces its IP addresses by BGP from several points and relies on BGP routing to deliver packets to the closest frontend [2].

- **Programming and automation.** The job advertisement asks for software development experience in one or more programming languages [3]. Google's Zero Touch Prod requires every production change to come from automation, to be prevalidated by software, or to go through an audited breakglass mechanism [4]. Google estimates it could have prevented or mitigated about 13% of the outages it evaluated [4].

- **Version control and testing.** Many Google teams keep service configuration in version control and change it through reviewed changelists [2]. Google advocates integration testing because unit tests do not fully replicate the runtime environment [5]. In Google's experience, a majority of incidents are triggered by binary or configuration pushes [2].

- **Reliability measurement.** An SLI is time series data showing how good the level of service is [6]. SLOs are reliability targets measured by SLIs [6]. At Google a well-thought-out and adopted SLO is key to prioritizing reliability work, and toil is capped at 50% of an SRE's time [2].

- **Troubleshooting distributed systems.** A Google case study shows new memory limits in Borg, Google's cluster operating system (Section 5), that ignored small RAM spikes, so Borg killed tasks that went slightly over and caused local disruption and latency [7]. Google's Disaster Recovery Training (DiRT) exercises simulate regional outages [2].

3.2 Soft skills

- **Blameless communication.** The Incident Management Guide calls open and blameless postmortem writing the most effective tool Google has found for learning from incidents [8]. SRE promotes blamelessness instead of citing "human error" [9].
- **Psychological safety and teamwork.** A team in control of major systems must feel psychologically safe and be able to say no to peers and leaders without fear of retribution [9]. Communication, agreement and trust are paramount to healthy SRE and developer relationships [10]. One SRE says their team drops what it is doing to help an on-caller who needs it [11].
- **Incident roles and calm under pressure.** Incident roles are based on knowledge and incident context, not reporting chains [8]. Google SRE advises practicing recovery mechanisms beforehand to keep your cool in a high-stress situation [6].
- **Managing workload.** Google says heroism should not be relied on in the long term, and that on-call done right does not need constant heroism [6]. Too much on-call can burn SREs out, and too little makes them lose familiarity with the service [10].

Why these matter. Our reading, which is an interpretation and not a sourced claim, is that SREs work in pairs and teams during incidents and pass their knowledge on through written documents. Clear speech, careful writing and trust are therefore part of the job and not extras. We found no source that covers presentation skills or time management for SREs. For English, we found only that the internship posting asks for a CV and a transcript in English (Section 4).

4 Selected Organization and Working Environment

Background and services. Google's SRE book says that SREs at Google operate services such as planet-scale storage, email for hundreds of millions of users and web search, which is where Google began [1]. The "site" in SRE first meant keeping the google.com website running, and SRE now also runs services that are not websites, from internal infrastructure such as Bigtable to Google Cloud Platform for external developers [1]. The book names Search, Ads, Gmail and Photos as services whose reliability targets Google analyzes [1]. Google's own company page says the Google story begins in 1995 at Stanford University, that Google Inc. was officially born in August 1998 after Sun co-founder Andy Bechtolsheim wrote a check for $100,000, and that Google has since moved to its current headquarters, the Googleplex, in Mountain View, California [12]. It says Google makes hundreds of products used by billions of people [12]. Alphabet, the parent of Google, says in its June 2026 investor presentation that its annual revenue more than doubled from 2020 to 2025, to over $400 billion, and that 13 products have more than a billion users, five of them more than 3 billion [13]. The only customer count we found is a May 2023 Google Workspace post that gives 3 billion users and 9 million paying customers for Workspace alone [14].

How SRE is organized. SRE and product development are separate organizations at Google, each with its own focus, priorities and management [2]. Product teams own their service by default, and SRE is not available or warranted for most services [2]. Product teams fund the growth of SRE with new hires when a product succeeds [2]. SRE has infrastructural teams, service teams and horizontal product teams [1]. It is not a command-and-control organization, and because of emergency response and pager rotations it is spread over at least a few time zones [1]. Google has hundreds of on-call rotations, and many developer teams, not only SRE, are on call for their own service [2]. The Workbook also says the efficacy of SRE at Google is tied to Google's widely accessible unified codebase [2]. The books give neither the size of the SRE organization nor its office locations.

Working environment and culture. SRE at Google inherited much of its culture from the surrounding company [2]. Google says it assumes failure can happen at any time, and that it does not get its culture right all the time [4]. Google prefers multi-site teams in part because night shifts harm

people's health [1], and it compensates on-call work with time off or cash, capped at a proportion of overall salary [2]. All new SREs attend SRE EDU, a weeklong course on key concepts, tooling and platforms, and can later spend a week with another team through SRE Exchange [2]. The books do not describe offices, working hours, remote-work policy or pay beyond on-call compensation.

Opportunities for students. The books mention interns only in two case studies: an SRE team that expected three interns to help its workload [2], and the Chrome security team, which has converted interns to full-time employees [4]. Engineers from product development can spend six months in an SRE team through the Mission Control program [2]. Google's careers site has a Students section with internships, early-career roles and apprenticeships [15]. Its 2027 posting for a Software Engineering, Site Reliability Engineering BS/MS Intern in London is the SRE internship we found [16]. It is intended for students in their penultimate or final year of a full-time Bachelors or Masters degree in the EMEA region (Europe, Middle East and Africa), who must be located in the UK for the program [16]. The internship lasts at least 13 weeks in summer 2027, applications close on 23 October 2026, and Google says applicants are not eligible for immigration sponsorship [16]. Applicants need a CV and a transcript in English [16]. The Site Reliability intern works on projects such as troubleshooting traffic anomalies, monitoring and alerts, and building automation, and Unix/Linux is among the preferred experience [16]. In our reading, a student in Thailand falls outside the EMEA condition of this posting. We found no SRE internship posting open to students in our region.

5 Operating Systems Required or Recommended

The sources do not say why Google chose Linux. They do describe Linux properties that, in our reading, matter for SRE work, and we present those.

Production servers. The Linux kernel underpins much of Google's production infrastructure [4]. Google has an internal Linux distribution. It first installed every datacenter machine with a "golden" base image and later moved to separate release units for each software package [4]. A 2013 paper by a Google engineer describes a live upgrade of Google's servers from a Red Hat 7.1 image to a Debian Testing based distribution built from source, without a long "flag day", with applications running in a different partition from the base distribution [17]. Resource allocation in the datacenters is handled by Borg, which the SRE book calls Google's cluster operating system [1]. The Workbook says Borg is Google's internal container management system and runs huge numbers of Linux containers [2].

Workstations. A 2022 Google Cloud Blog post says Google operates many OS platforms for its employees, including a Linux system, across hundreds of thousands of devices [18]. Its internal Linux distribution, Goobuntu, was for a long time based on Ubuntu LTS. In 2018 Google completed a move to gLinux Rodete, short for Rolling Debian Testing, because Debian offered a smooth in-place migration. Each release is guided to the fleet with SRE practices such as incremental canarying, and the rolling schedule lets Google patch security holes on the whole fleet quickly [18]. No source says which OS Google SREs use on their own workstations. Our recommendation, which is an assumption, is to prepare to work every day in a Linux shell, both on a workstation and on remote servers.

Kernel updates and logs. Google regularly pushes new kernels to its whole fleet, with a target of less than 30 days [4]. It uses ksplice, a runtime kernel patch that avoids a reboot, and SREs applied one to production systems for two 2018 kernel vulnerabilities [4]. The Linux kernel documentation describes livepatching the same way [19]. For investigations, Linux provides syslog and auditd logs [4].

Isolation and resource limits. Containers share the OS kernel [20]. Linux cgroups organize processes in a hierarchy and distribute system resources along it, and Google contributed cgroups to the Linux kernel [19, 20]. When a cgroup reaches its `memory.max` limit and the memory cannot be reduced, the kernel's out-of-memory killer runs inside that cgroup [19]. Google has many out-of-memory conditions every day [4], and Borg kills and restarts a task that uses more resources than it requested [1]. A server can also run out of file descriptors or process IDs [1]. For untrusted code,

Google added ptrace sandboxing to App Engine because the Linux kernel exposed a large attack surface [4], and its security overview lists Linux user separation and an application kernel such as gVisor among its isolation techniques [21].

6 Hardware Requirements

Google datacenters. The SRE book, which carries a 2017 copyright, says that most of Google's compute resources are in Google-designed datacenters with proprietary power distribution, cooling, networking and compute hardware [1]. Google designs its server boards and networking equipment [21] and custom builds servers exclusively for its data centers [22]. Tens of machines sit in a rack, and a datacenter building usually houses several clusters [1]. The SRE book's main text says the compute hardware is the same across the board, but a footnote says some datacenters end up with multiple generations [1], and the load-balancing chapter agrees that machines in one datacenter are not necessarily the same [1]. Hardware failure is routine: in a single cluster in a typical year, thousands of machines fail and thousands of hard disks break [1]. The SRE book also says that Google's Jupiter network connects hundreds of Google-built switches in a Clos fabric and supports 1.3 Pbps of bisection bandwidth in its largest configuration [1]. The B4 backbone connects datacenters to each other [1]. The lowest layer of the storage stack uses both spinning disks and flash [1]. Besides its own datacenters, Google has racks of proxy and cache machines in colocation facilities [2].

Firmware and processors. Google manages the firmware of its machines and network cards with the same systems it uses for host software, and names Arm and x86 CPUs among its environments [4]. It designs custom chips, including the Titan hardware security chip, which serve as hardware roots of trust [21].

Sizing numbers are exercise assumptions. The SRE Workbook's design exercise assumes a standard machine of 16 cores, 64 GB of RAM and 1 Gbps of network throughput, and a common 4 TB disk sustaining about 200 operations per second [2]. These are assumptions of the exercise, not a statement of Google's hardware.

Hardware for practice (recommendation). Debian's installation guide lists 512 MB of RAM as the minimum and 1 GB as recommended, with 4 GB of disk, for an install without a desktop [23]. The Ubuntu Server basic-installation tutorial recommends at least 2 GB of RAM and 5 GB of disk [24]. Minikube needs 2 CPUs, 2 GB of free memory and 20 GB of free disk [25]. From these minimums we estimate, as an assumption and not a source's advice, that a personal laptop or desktop with about 8 GB of RAM can run either minikube or two or three small Linux virtual machines (Debian or Ubuntu Server) without a desktop, and that 16 GB runs both at once. Server-class hardware is not needed.

7 System Architecture and Infrastructure

The SRE book describes the infrastructure of Google's production environment as it was when the book was written. We describe it layer by layer and do not claim that it is unchanged today.

Cluster management and virtual machines. Borg starts a job by finding machines for its tasks and telling them to start the server program. A malfunctioning task is killed and restarted, possibly on another machine [1]. Borg accounts for failure domains, so it will not run all tasks of a job on one rack [1]. For virtual machines, Google decided that running each user's code of the original App Engine in an independent virtual machine was too inefficient for the planned adoption [4]. Running two virtual machines of mutually distrustful parties on the same hardware carries a certain risk [4].

Storage and consensus. Colossus creates a cluster-wide filesystem with replication and encryption, and Bigtable is a NoSQL database that can handle petabyte-size databases [1]. The Chubby lock service handles locks across datacenters and uses the Paxos protocol [1]. The SRE book recommends formally proven and thoroughly tested distributed consensus systems for leader election, critical shared state and distributed locking [1].

Network, request path and load. The SRE book says that Google uses an OpenFlow-based software-defined network [1]. Its Global Software Load Balancer balances load on three levels, and all services talk to each other through an RPC infrastructure called Stubby [1]. The Google Front End (GFE) is a frontend layer for most Google services. It terminates incoming HTTP(S), TCP and TLS traffic, provides DDoS countermeasures, and routes and load balances traffic [4]. GFE can also drop or absorb connections when a service is overloaded [4]. A component should serve errors for excessive requests instead of crashing, because a crash removes all of its capacity [4]. However efficient load balancing is, some part of a system will eventually be overloaded [1].

Backup and recovery. Replication and redundancy are not recoverability [1]. The focus of backups should be data recovery, which separates real backups from archives [1]. In a 2011 Gmail incident, Google recovered lost data from its previously undisclosed tape backup system [1]. Google also runs annual DiRT exercises [4].

Practice environment (recommendation). A typical practice environment for a student is a small Linux virtual machine. That is our recommendation and not evidence about Google.

8 Software, Tools and Applications

Languages and build tools. Google's build tool Blaze, open sourced as Bazel, builds binaries from what Google calls its standard languages: C++, Java, Python, Go and JavaScript [1]. Builds are hermetic, meaning they depend on known versions of tools such as compilers and not on software installed on the build machine [1]. On scripting, the SRE book describes the shell scripts Google first used to configure clusters as brittle and says the initial automation was simple Python scripts [1].

Editors, compilers and IDEs. Compiler plug-ins such as Error Prone for Java and Tsetse for TypeScript can prohibit risky code patterns at compile time, and compiler errors give faster feedback than linters or checks at code review time [4]. We found no book that names an editor or an IDE as an SRE tool.

Debugging and profiling. Every Google server has an HTTP server that provides diagnostics and statistics for debugging [1]. Valgrind catches memory errors without recompiling, AddressSanitizer detects buffer overflows and use after free, and ThreadSanitizer detects data races [4]. These claims do not show that Google SREs use these tools every day.

Version control, containers and shell. The SRE book says that Google stores all of its schemes for distributing configuration in the primary source code repository and enforces a strict code review requirement [1]. The SRE Workbook calls checking configuration files into a versioning system such as Subversion or Git common practice [2]. It describes Kubernetes as an open source system for orchestrating containerized workloads on a cluster of machines, used by writing YAML files and running kubectl [2]. The SRE book calls Kubernetes Borg's descendant, started by Google in 2014 [1]. Much of Google's distributed automation once relied on SSH, which was clumsy from a security perspective because people needed root on many machines, so Google replaced sshd with an authenticated, ACL-driven, RPC-based Local Admin Daemon [1]. *Building Secure and Reliable Systems* says that, instead of setting services up by hand, you might write imperative turnup automation or use a declarative configuration language such as Terraform [4].

Release engineering and CI/CD. The SRE book says that Google has an automated release system called Rapid, that some teams use a "Push on Green" model in which they deploy every build that passes all tests, and that a continuous test system runs unit tests against the mainline each time a change is submitted [1]. The Workbook defines canarying as a partial and time-limited deployment of a change and its evaluation [2]. *Building Secure and Reliable Systems* describes the software supply chain as writing, building, testing and deploying a system through version control, a continuous integration (CI) pipeline and a continuous delivery (CD) pipeline, and names Jenkins as an example [4]. In one

Workbook case study, a small group at Waze, which Google acquired in 2013, proposed continuous deployment with Spinnaker. This is one team's case and not general Google practice [2].

Monitoring, alerting and incident response. The SRE book says that Google built its monitoring system Borgmon shortly after it created the Borg job scheduler in 2003. Borgmon fetches the `/varz` URI of each monitored target at set intervals, and the idea of using time-series data to generate alerts is available outside Google through open source tools such as Prometheus [1]. The Workbook strongly recommends treating monitoring configuration as code [2]. The SRE book names latency, traffic, errors and saturation as the four golden signals and Dapper as a tool for tracing requests [1]. Alertmanager routes alert notifications, page-worthy alerts go to the on-call rotation and subcritical ones go to ticket queues [1]. The Workbook names three main roles in incident response: the Incident Commander, the Communications Lead and the Operations Lead [2].

Server and database software. Google's Ads database, a MySQL deployment, ran on Borg on shared machines that were rebooted for kernel upgrades [1]. The report *Creating a Production Launch Plan* gives the example of a relational database with an upper limit on transaction throughput, where adding CPU or memory scales only so far [26]. Caches sometimes become hard dependencies of a service [1]. A message queue that stores data only in volatile memory can lose transactions [4], and in the Waze case study the message queueing system was critical because every component used it [2]. This section does not cover web servers such as Apache or Nginx, or the operation of PostgreSQL, MongoDB or Redis, because we have no supported claim on them.

9 Relationship to Operating Systems Course Knowledge

Table 1 connects course concepts to SRE work. The middle column holds facts from documentation. The right column holds our recommended ways to use them in an investigation. These recommendations are assumptions and not evidence of how Google works.

Table 1: OS concepts and their use in SRE work.

Concept	Fact	SRE use (our recommendation)
Processes, threads, scheduling	The Linux scheduler selects which runnable thread runs next [27]. Threads of one process share data and heap but have their own stacks [27].	For a slow multithreaded service, compare runnable threads with latency, and tell ready-queue waits from event waits. For a restart, read process logs and signals before naming a cause.
Virtual memory	Page tables translate virtual to physical addresses [19]. A major page fault needs I/O [28], and direct reclaim can stall an allocation [19].	Compare a service's virtual mappings with its resident memory. Read changes in major-fault counts as evidence, not as proof of exhaustion.
File systems and I/O	A successful `write()` does not mean the data is on disk [27]. `fsync()` waits until the device reports completion [27].	When investigating latency or crash recovery of a service that writes data, separate successful writes, cache activity and completed synchronization.
Networking and permissions	Sockets connect processes to the kernel network stack, TCP retransmits lost packets, and missing directory search permission gives EACCES [27].	For a slow or failed connection, check its socket and TCP behavior. For a file-access failure, check directory permissions before widening them.

Concept	Fact	SRE use (our recommendation)
Containers, virtualization and resource control	Namespaces implement containers [27]. Cgroups distribute resources [19]. Containers share one kernel [29]. Kubernetes enforces CPU limits by throttling and memory limits by out-of-memory kills [30]. QEMU supplies a virtual machine with its own guest OS [31].	Compare the guest OS and virtual hardware of a virtual machine with the shared kernel of containers, and separate namespace isolation from cgroup resource control.
Failure detection and recovery	Kubernetes can restart unhealthy containers or stop traffic to containers that are not ready [30]. The SRE book recommends limiting retries per request [1].	Separate an observed failed check from a confirmed root cause, and test bounded retries before assuming they restore availability.

A recommended chain. In the form of the assignment's examples, our recommended chain is: SRE, then a hypothetical containerized web service, then Linux, then a server or virtual machine that provides CPU, RAM, storage and network, then Docker containers managed by Kubernetes, then processes, scheduling, virtual memory, file I/O, networking, permissions and resource isolation. This is an example for study and not a verified description of Google's infrastructure. Figure 1 (page 9) shows a different chain: the Google-specific one, drawn from our sources, with two labeled assumptions.

10 Hands-On Preparation Plan

This plan is our recommendation and an assumption. Documentation facts that support single steps carry citations.

Lab environment. Use a disposable local virtual machine with Ubuntu Server 24.04 LTS. We chose Ubuntu because the guides we cite for SSH and Docker are Ubuntu guides, and because Google's own workstation distribution was based on Ubuntu until 2018 [18] (Section 5). Debian, the base of that distribution since 2018, would also work. Start with 2 vCPUs, 4 GB of RAM and 40 GB of disk, on a host with at least 8 GB of RAM. The RAM and disk figures are above the Section 6 minimums, to leave room for containers, and the CPU figure equals the minikube minimum. Complete the installation, the package updates, and the user, permission and network configuration. SSH supports key authentication instead of a password [32], so finish this step with an SSH key login. Then add Git with Bash or Python, Docker Engine and Prometheus, in this order. Add minikube and kubectl only after the core lab passes [25].

Roadmap with completion gates. Advance to the next gate only when the current gate passes.

1. Show an SSH login to the lab, and inspect processes, memory, files and sockets.
2. Keep a small HTTP service and a Bash or Python health-check script in Git, and test healthy and failed responses. A Git commit records a snapshot that can later be reverted to or compared [33].
3. Containerize the service, and show a reproducible startup, an HTTP request and the logs. Docker's Ubuntu guide verifies an installation by running the hello-world image [34].
4. Scrape the service metrics with Prometheus, which collects metrics from HTTP endpoints [35]. Record the four golden signals [1] and test a documented practice alert.
5. Stop the service or introduce a permission error. Document detection, evidence, mitigation and recovery, and write a postmortem with impact, timeline, contributing causes and a preventive action. The SRE book recommends regular review sessions for postmortem drafts [1].

As an optional extension, deploy the same service to a local minikube cluster and reproduce a configuration change and a recovery.

Deliverables. Keep a reproducible README, the versioned lab configuration, the health-check script, the monitoring evidence and a reviewed postmortem. Ask a peer to reproduce the setup and review the incident explanation. The SRE Workbook cautions that automation should not remove human understanding of what is going wrong [2].

11 Use of Generative AI Platforms/Models

We used two Generative AI platforms: Claude Code (Anthropic) [36–38] and Codex (OpenAI) [39]. We chose them because both are agent harnesses: tools that let a model read files, run scripts and search the web under rules that we wrote in the repository, so that the model does the right job at each step. One team member used Codex for their research topics, and the other members used Claude Code. Codex ran claim research and the blind author check only. Claude Code also ran the reviewer rechecks and drafted this report. The same rules applied to both platforms. We did not compare the

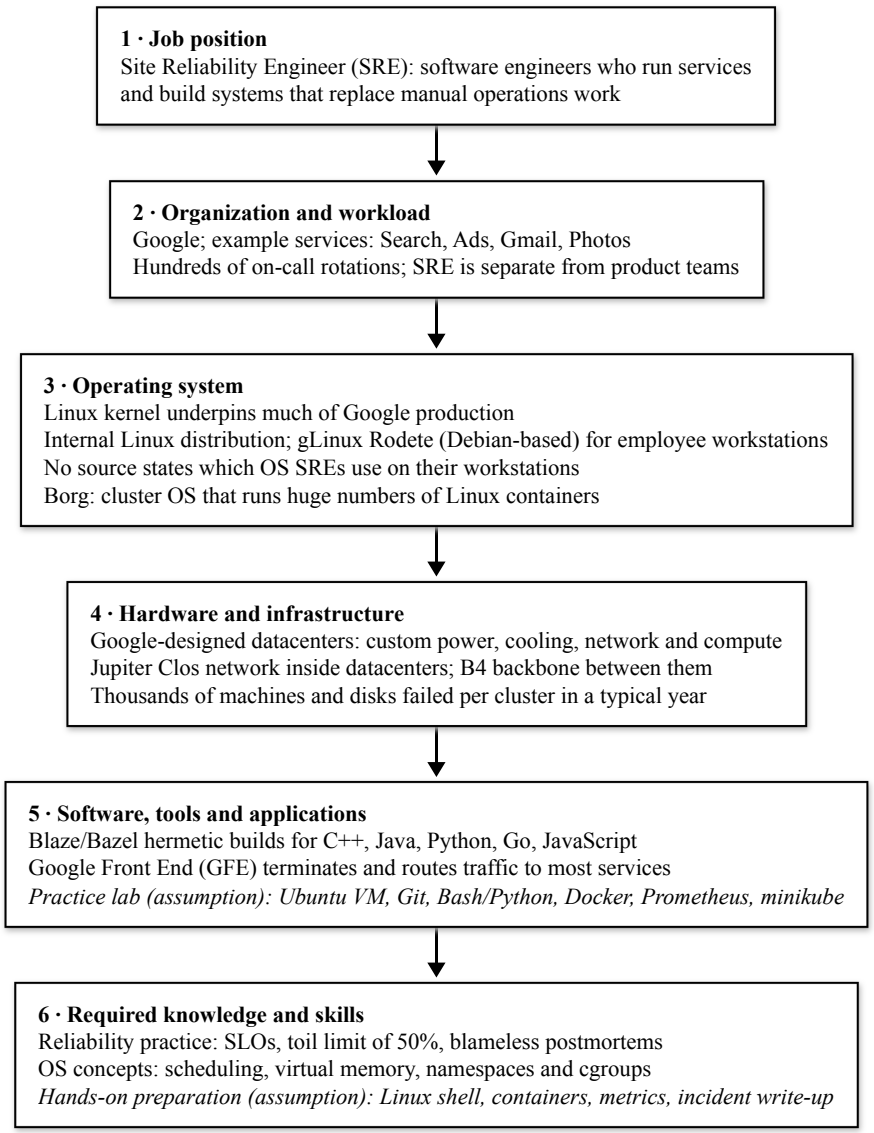


Figure 1: From job position to required knowledge and skills for an SRE at Google. Most sources date from 2017 to 2020. Each stage rests on claims from the cited sources [1, 2, 4, 8, 18, 19, 27]. The arrows read: works at, runs on, sits on top of, is operated with, requires. The two italic lines are our assumptions, a typical practice environment and a preparation recommendation, not verified facts about Google. The link from the OS concepts to the job is our interpretation.

quality of their output, and because different people used them on different topics, we cannot say which platform is better. Table 2 lists each platform and model, what we used it for, and how many prompt logs it produced. The complete record is 85 prompt logs in our public project repository [40]; we marked 5 of them as significant, the ones that matter most for this report, and the Appendix shows the prompts we typed in them.

Table 2: GenAI platforms and models used in 2026, from the prompt logs.

Platform and model	Used for	Logs	Period
Claude Code, Claude Opus 5.5 (claude-opus-5-5)	claim research, blind author check, reviewer recheck	33	10-09 to 10-15
Claude Code, Claude Opus 5.5 and Claude Sonnet 5.5	claim research	1	10-07
Claude Code, Claude Sonnet 5.5 (claude-sonnet-5-5)	claim research, blind author check, reviewer recheck, report drafting	41	10-06 to 10-16
Codex, gpt-6.1-sol	claim research, blind author check	10	10-10 to 10-12

How we used GenAI. GenAI helped us read and organize sources. It did not decide what is true. Each factual statement in our knowledge base is a claim with a published source, an access date and an exact sentence copied from that source. A claim could pass through four steps:

1. **Research.** An agent read published sources (mainly the Google SRE books and official documentation) and proposed claims with a quote.
2. **Blind author check.** A fresh agent received only the claim text and the source address, never the stored quote, and said whether the source supports the claim.
3. **Reviewer recheck.** A different team member ran the same blind check in a separate session.
4. **Human review.** The reviewer opened every cited source. Only then did the claim become "human-verified".

Of 522 claims in our knowledge base, 372 passed all four steps and are human-verified. Another 132 passed the blind author check but not the human review. This report draws on both groups. A further 18 are assumptions and recommendations. They have no source by design, are labeled as such, and can never become human-verified.

What we corrected. The checks changed the content. A reviewer recheck found that the phrase "not a fixed limit" went beyond what the SRE book page says, and we replaced the claim with one that stays inside the source. A recheck of 24 claims marked 10 as unreachable because the source site could not be opened, and we rechecked those 10 using archived and mirrored copies. After a review of one pull request, we dropped, reworded and retyped claims about server and database software. Reworded claims went through the blind check again; in one recheck, two of three claims were supported and one only partly.

Report creation. We also used GenAI to draft report text and to create the relationship diagram in Figure 1. A fresh agent that had none of our context checked the report against the assignment requirements. Our rules forbid sending personal or confidential data to a GenAI service. The claim checks received only a claim and a public web address, and the logs mark removed private spans with `[REDACTED]` tags. The Appendix shows the prompts that we typed for this work. Claude Code wrote the instructions for the checking agent itself, and the logs do not hold them.

12 Conclusion

Figure 1, on page 9, shows how our findings fit together, from the job position to the knowledge it requires.

What we found. At Google, SRE means software engineers who run services and build systems to replace manual operations work, with operational work, or toil, capped at 50% of an SRE's time [1,2]. According to the books, the Linux kernel underpins much of Google's production infrastructure [4], and Borg, the cluster operating system, runs huge numbers of Linux containers [1,2]. It runs in Google-designed datacenters where, in the SRE book's account, thousands of machines and disks failed in a typical year in one cluster [1]. Containers rely on Linux namespaces and control groups and share one kernel [19, 27, 29]. In our reading, this links the job to process management, scheduling, virtual memory and resource control.

What we could not find. In the sources we read, we found no statement of which operating system Google SREs use on their own workstations. We found no total count of Google's business customers, only a 2023 figure for Google Workspace. The one SRE internship posting we found is limited to students in the EMEA region, and we found no SRE internship open to students in Thailand. We have no supported claim on web servers such as Apache or Nginx, or on running PostgreSQL, MongoDB or Redis. We also found no source on the specific projects given to student SREs beyond the general description in that posting. If none is open to us, we would look for SRE or DevOps internships elsewhere, which is an assumption. These gaps are noted where they arise. Anything about Google's internal environment that no source states is labeled as an assumption.

How GenAI can help in preparation and in the job. This is our recommendation and an assumption, drawn from how we used GenAI here. In preparation, GenAI can explain an unfamiliar command or log line and draft a README or postmortem outline for the lab. On the job, SRE teams write postmortems after incidents [1], and GenAI can summarize a long log or timeline and draft the first version. It cannot confirm a root cause, because it does not see the real system, and our own checks found claims that went beyond their source. The SRE Workbook cautions that automation should not remove human understanding of what is going wrong [2]. We therefore recommend GenAI for explanations, summaries and first drafts, with every statement checked against the logs, metrics and documentation. OS knowledge makes that check possible, because it lets a reader tell a correct explanation of processes, memory or sockets from a wrong one.

Reflection. GenAI sped up reading and organizing sources, but it made mistakes that human review had to catch. We kept Generative AI output as fact only where a source supported it, and we labeled the rest as recommendations or assumptions.

References

[1] Site Reliability Engineering. Edited by B. Beyer, C. Jones, J. Petoff and N. R. Murphy. Published by O'Reilly Media, Inc., copyright 2017. 30 chapters read at `https://sre.google/sre-book/`; the chapter of each claim is in the project repository. Accessed: 2026-10-06 to 2026-10-14.

[2] The Site Reliability Workbook. Edited by B. Beyer, N. R. Murphy, D. K. Rensin, K. Kawahara and S. Thorne. Published by O'Reilly Media, Inc., copyright 2018. 19 chapters read at `https://sre.google/workbook/`; the chapter of each claim is in the project repository. Accessed: 2026-10-07 to 2026-10-14.

[3] Software Engineer, Android SRE, Google Careers job ad. `https://www.google.com/about/careers/applications/jobs/results/121141635160384198-software-engineer-android-sre`. Accessed: 2026-10-11.

[4] Building Secure and Reliable Systems. By H. Adkins, B. Beyer, P. Blankinship, A. Oprea, P. Lewandowski and A. Stubblefield. Launched in April 2020. 14 chapters read at `https://google.github.io/building-secure-and-reliable-systems/raw/`; the chapter of each claim is in the project repository. Accessed: 2026-10-06 to 2026-10-14.

[5] Lessons Learned from Two Decades of Site Reliability Engineering (Google SRE 20th anniversary PDF). https://sre.google/static/pdf/LessonsLearnedFromTwoDecades.pdf. Accessed: 2026-10-12.

[6] Google SRE website articles. Articles read at https://sre.google/resources/: practices-and-processes/measuring-reliability, practices-and-processes/no-heroes, practices-and-processes/product-focused-reliability-for-sre, practices-and-processes/slo-adoption-and-usage, practices-and-processes/sre-fundamentals-course, practices-and-processes/twenty-years-of-sre-lessons-learned. Accessed: 2026-10-07 to 2026-10-09.

[7] Case Studies in Infrastructure Change Management: How Google Rebuilds the Jet While Flying It (O'Reilly/Google report). https://sre.google/static/pdf/CaseStudiesInfrastructureChangeManagement.pdf. Accessed: 2026-10-09.

[8] Google SRE: Incident Management Guide (PDF). https://sre.google/static/pdf/IncidentManagementGuide.pdf. Accessed: 2026-10-07.

[9] Enterprise Roadmap to SRE (Google Cloud/SRE report). https://sre.google/static/pdf/enterprise-roadmap-to-sre.pdf. Accessed: 2026-10-07.

[10] Training Site Reliability Engineers (O'Reilly report by Google authors, 2019/2020). https://sre.google/static/pdf/TrainingSiteReliabilityEngineers.pdf. Accessed: 2026-10-12.

[11] Google SRE Prodcast transcript: Incident Response with Sarah Butt and Vrai Stacey. https://sre.google/prodcast/transcripts/sre-prodcast-03-06/. Accessed: 2026-10-07.

[12] How we started and where we are today - Google - About Google. https://about.google/company-info/our-story/. Accessed: 2026-10-16.

[13] Alphabet investor presentation: June 2026. https://blog.google/alphabet/investor-presentation-june-2026/. Accessed: 2026-10-16.

[14] Build workflows across Google Workspace to connect user experiences across apps (blog post, May 2023). https://workspace.google.com/blog/product-announcements/workflows-across-workspace. Accessed: 2026-10-16.

[15] Students - Google Careers. https://www.google.com/about/careers/applications/students/. Accessed: 2026-10-16.

[16] Software Engineering, Site Reliability Engineering BS/MS Intern, 2027 (London), Google Careers. https://www.google.com/about/careers/applications/jobs/results/100028133205254854-software-engineering-site-reliability-engineering-bsms-intern-2027. Accessed: 2026-10-16.

[17] Marc Merlin (Google), Live Upgrading Thousands of Servers from an Ancient Red Hat Distribution to 10 Year Newer Debian Based One, USENIX LISA '13. https://www.usenix.org/conference/lisa13/technical-sessions/presentation/merlin. Accessed: 2026-10-08.

[18] Google Cloud Blog. Posts read at https://cloud.google.com/blog/: products/gcp/7-ways-we-harden-our-kvm-hypervisor-at-google-cloud-security-in-plaintext, topics/developers-practitioners/how-google-got-to-rolling-linux-releases-for-desktops. Accessed: 2026-10-08.

[19] The Linux Kernel documentation. Pages read at https://docs.kernel.org/: admin-guide/cgroup-v2, admin-guide/mm/concepts, livepatch/livepatch, mm/page_tables, scheduler/sched-stats. Accessed: 2026-10-08 to 2026-10-12.

[20] Google Cloud, What are containers? https://cloud.google.com/learn/what-are-containers. Accessed: 2026-10-08.

[21] Google Cloud, Google infrastructure security design overview. https://docs.cloud.google.com/docs/security/infrastructure/design. Accessed: 2026-10-08.

[22] Google Data Centers, homepage. https://datacenters.google/. Accessed: 2026-10-08.

[23] Debian GNU/Linux Installation Guide, 3.4 Meeting Minimum Hardware Requirements. https://www.debian.org/releases/stable/amd64/ch03s04.en.html. Accessed: 2026-10-08.

[24] Ubuntu Server documentation: Basic installation. https://ubuntu.com/server/docs/tutorial/basic-installation/. Accessed: 2026-10-11.

[25] minikube documentation, minikube start. https://minikube.sigs.k8s.io/docs/start/. Accessed: 2026-10-11.

[26] Creating a Production Launch Plan: For a Successful Launch, Planning Isn't Optional (O'Reilly/Google report). https://sre.google/static/pdf/CreatingProductionLaunchPlan.pdf. Accessed: 2026-10-07.

[27] Linux man-pages. Pages read at https://man7.org/linux/man-pages/: man2/fsync.2, man2/mmap.2, man2/write.2, man7/namespaces.7, man7/path_resolution.7, man7/pthreads.7, man7/sched.7, man7/signal.7, man7/socket.7, man7/tcp.7. Accessed: 2026-10-10.

[28] Linux man-pages: getrusage(2), published by Debian. https://manpages.debian.org/testing/manpages-dev/getrusage.2.en.html. Accessed: 2026-10-11.

[29] Docker Docs: What is a container? https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/. Accessed: 2026-10-10.

[30] Kubernetes documentation. Pages read at https://kubernetes.io/docs/: concepts/configuration/manage-resources-containers, concepts/workloads/pods/probes. Accessed: 2026-10-10 to 2026-10-11.

[31] QEMU documentation: System Emulation Introduction. https://www.qemu.org/docs/master/system/introduction.html. Accessed: 2026-10-10.

[32] Ubuntu Server documentation: OpenSSH server. https://ubuntu.com/server/docs/how-to/security/openssh-server/. Accessed: 2026-10-11.

[33] Pro Git, second edition: Recording Changes to the Repository. https://git-scm.com/book/en/v2/Git-Basics-Recording-Changes-to-the-Repository. Accessed: 2026-10-11.

[34] Docker Docs: Install Docker Engine on Ubuntu. https://docs.docker.com/engine/install/ubuntu/. Accessed: 2026-10-11.

[35] Prometheus documentation: Getting started. https://prometheus.io/docs/prometheus/latest/getting_started/. Accessed: 2026-10-11.

[36] Anthropic. Claude Code, Claude Opus 5.5 (claude-opus-5-5). GenAI platform. Used for claim research, blind author check, reviewer recheck. Accessed: 2026-10-15.

[37] Anthropic. Claude Code, Claude Opus 5.5 and Claude Sonnet 5.5. GenAI platform. Used for claim research. Accessed: 2026-10-07.

[38] Anthropic. Claude Code, Claude Sonnet 5.5 (claude-sonnet-5-5). GenAI platform. Used for claim research, blind author check, reviewer recheck, report drafting. Accessed: 2026-10-16.

[39] OpenAI. Codex, gpt-6.1-sol. GenAI platform. Used for claim research, blind author check. Accessed: 2026-10-12.

[40] Knowledge base and prompt logs for the SRE career and OS research. https://github.com/ilma-org/os-coop-job-research. Accessed: 2026-10-17.

Appendix A Complete GenAI Prompts

These are the prompts that we typed to the GenAI platforms in the significant interactions, in order and word for word. Each prompt shown is complete, and none is shortened. [REDACTED:kind] marks a span removed for privacy. A reply of 12 characters or fewer, such as "yes", is not shown and is not

counted as significant. A turn that gave only a routine instruction, such as to fix a layout detail, find a file, report status or run a git or sign-off step, and shaped no report text, claim or decision, is not shown either. Each prompt is labeled with its turn number in the full log, and when we sent several messages before the platform replied, the platform's log joins them under one turn number with a line of three hyphens. The count of shown prompts is given in each entry, and the full log holds every turn. Supporting data lists the files and web pages that we gave the model; files that the model opened itself are in the tool-call record of the full log. The replies and the tool-call records of each session are in the full log named in each entry, in the project repository [40].

A.1 Knowledge exploration

Knowledge exploration 1: Plan and research the topic 04, 05 and 06 claims (OS, hardware, architecture) from the Google SRE books and official Google sources. Claude Code (desktop app), Claude Opus 5.5 (claude-opus-5-5); 2026-10-09; log `2026-10-09-csinside-sre-books-04-06-research`; supporting data: repository file `docs/FinalAssignment_204341_1_69_AIAssignProject-1.pdf`; 1 web page (sre.google 1). Prompts (5 of 6):

Turn 1. @"[REDACTED:path]"
ยังไม่ต้องเขียนอะไร just plan สรุปมาอย่างง่ายว่าทำอะไรบ้าง หัวของที่เราได้เน้นทำคือ
04-operating-systems
05-hardware-requirements
06-system-architecture-infrastructure
หัวข้องอื่นจะเป็นของเพื่อน
เอกสารอ้างอิงหลักคือ https://sre.google/books/
อยากให้หาว่าควรไปเน้นอ่านที่ส่วนไหน วางแผนการจัดสรรเนื้อหา

Turn 2. writing clam 04 05 06 in 2 languages, Thai and English, separate file
Don't push Thai language file to git hub
But now don't push anything

Turn 3. 04 Are there any points where the information is unclear and requires external research?

Turn 4. Have you already looked for the additional information in the documents?

Turn 5. Discard the Thai-language files and continue the research; all information used for the write-up must be clear and factually verifiable.
[Request interrupted by user]
Discard the Thai-language files and continue the research; all information used for the write-up must be clear and factually verifiable.

Knowledge exploration 2: Research topic 11 OS mechanisms and proposed SRE connections. Codex, gpt-6.1-sol; 2026-10-10; log `2026-10-10-TonCS05-11-os-concepts-research`; supporting data: repository files `AGENTS.md`, `docs/assignment.md`, `docs/schema.md`, `knowledge-base/11-os-course-concepts/index.md`, `.agents/skills/research/SKILL.md`, `.agents/skills/agent-fact-check/SKILL.md`; 16 web pages (man7.org 10, docs.kernel.org 2, docs.docker.com 1, kubernetes.io 1, qemu.org 1, sre.google 1). Prompts (1):

Turn 1. งั้นหาข้อมูลในหัวข้อที่11แล้วบันทึกตามกติกาที่ได้ตั้งๆไว้

Knowledge exploration 3: Research topic 01 (SRE position and job description) into five notes of claims from the Google SRE books and one Google job ad. Claude Code (desktop app), Sonnet 5.5 (claude-sonnet-5-5); 2026-10-11; log `2026-10-11-bximai-01-position-research`; supporting data: 16 web pages (sre.google 15, google.com 1). Prompts (3 of 7):

Turn 1. Main Souce : https://sre.google/books/
start research on 01 . Split into 5 notes:
* role-and-duties
* daily-tasks-and-on-call

* typical-projects
* education-and-qualifications
* environment-overview.
Do one note at a time, starting with role-and-duties
Note : do not made up or guess.

Turn 2. Redo 01-06 by the rules: delete its block, re-add it with add_claim.py using the same claim text and quote, then update the claim IDs in Summary and Key points.
From now on, never hand-edit a claim block. If the wording is wrong, delete and re-add.

Turn 3. yes and always follow the instruction rules.

Knowledge exploration 4: Rework the uncommitted 03-soft-skills and 10-server-and-database-software claims and notes to the PR. Claude Code, Opus 5.5 (claude-opus-5-5); 2026-10-12; log `2026-10-12-nine14282-soft-skills-server-db-claims-rework`; supporting data: none. Prompts (2 of 3):

Turn 1. with this standard of writing. Let's look at my old uncommit work 03 and 10 if it need changed, if so, fix those change

Turn 2. verify with 2 sub agent again. one for word one for rule

A.2 Report creation

Report creation 1: Draft the LaTeX report from the knowledge base, add company and internship claims, build the cover-page template (the cover data stays local), and revise the report after a blind review. Claude Code (CLI v2.1.299), Claude Sonnet 5.5 (claude-sonnet-5-5); 2026-10-16; log `2026-10-16-nacs-970-report-drafting-and-review`; supporting data: repository files `docs/assignment.md`, `docs/FinalAssignment_204341_1_69_AIAssignProject-1.pdf`; 1 web page (google.com 1). Prompts (6 of 35):

Turn 1. list thing that can be add to document assignment

Turn 2. 1 i will add it later, we will use latex from now on, i will commit most of work on main now so update workflow
2 we select sre because we see oppuritunity in ai era that need site engineer since ai still can only do software stuff, google because big commpany and have delicate website on sre
3 look in prompt log
4 draft it for me
5 mermaid preview it for me first
6 mermaid
7 check repo
8 create it
---
recheck un verified topic
why 13 exisist is it really require by instuctor?

Turn 14. i recheck everything it's ok, but recheck for weak/example claim

Turn 22. 1 group website and book together, and appenddix only really importance log

Turn 24. in what we clound not found this is https://www.google.com/about/careers/applications/ look for internship program, search company history google, search for revenue or customer counts too. don't mention of no human sign off in report

Turn 35. start blind agent check of report is it match the instuctor requirement,
write a review